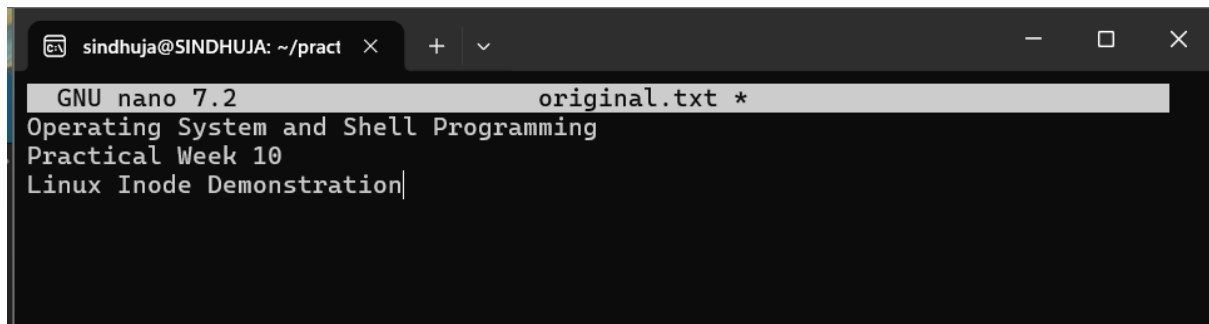


PRACTICAL WEEK 10

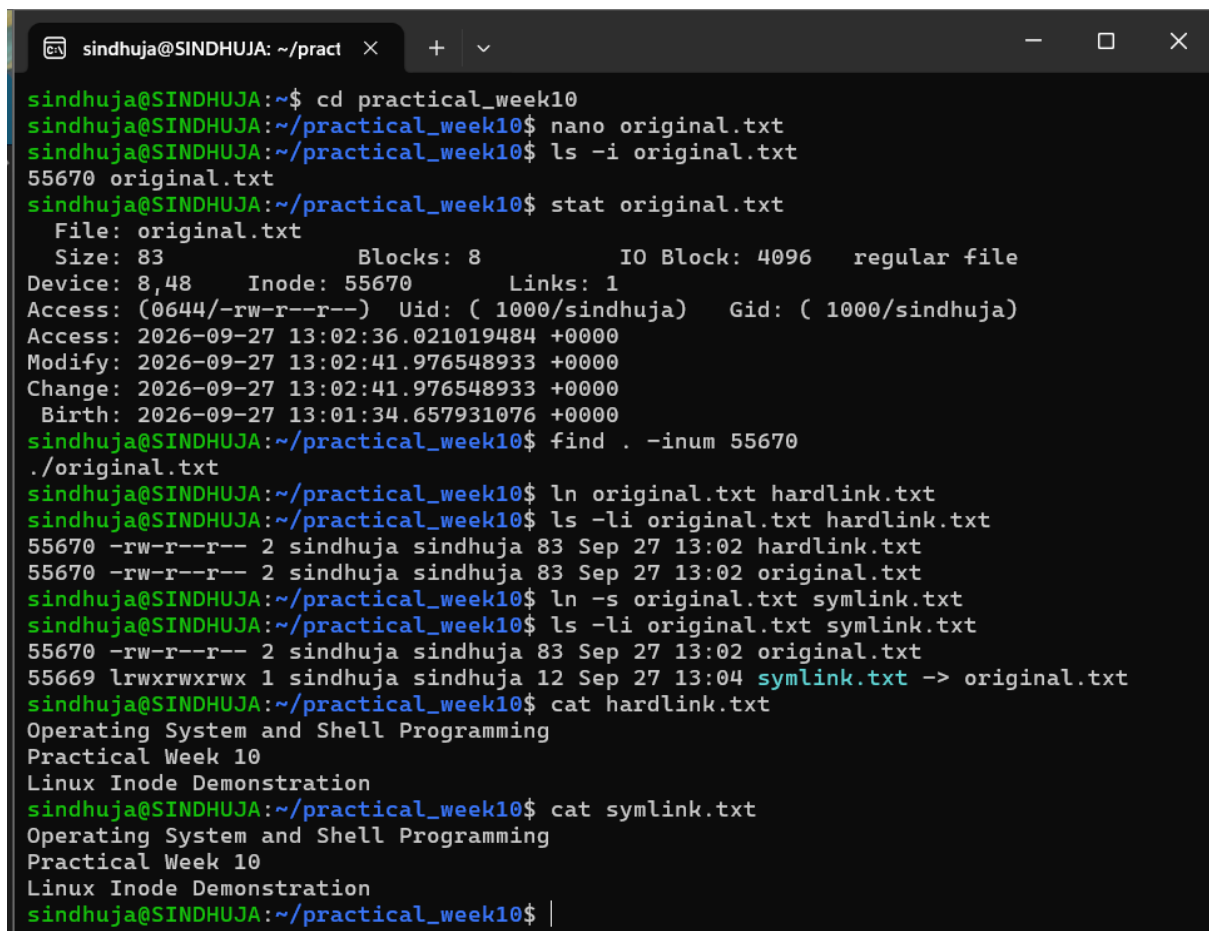
Aim:

To investigate inode structures using Linux commands (ls -li, stat, find), create hard links and symbolic links, and observe their impact on inode allocation.



```
sindhuja@SINDHUJA: ~/pract × + v
GNU nano 7.2 original.txt *
Operating System and Shell Programming
Practical Week 10
Linux Inode Demonstration|
```

Output:



```
sindhuja@SINDHUJA: ~/pract × + v
sindhuja@SINDHUJA:~$ cd practical_week10
sindhuja@SINDHUJA:~/practical_week10$ nano original.txt
sindhuja@SINDHUJA:~/practical_week10$ ls -li original.txt
55670 original.txt
sindhuja@SINDHUJA:~/practical_week10$ stat original.txt
  File: original.txt
  Size: 83          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 55670         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/sindhuja)   Gid: ( 1000/sindhuja)
Access: 2026-09-27 13:02:36.021019484 +0000
Modify: 2026-09-27 13:02:41.976548933 +0000
Change: 2026-09-27 13:02:41.976548933 +0000
 Birth: 2026-09-27 13:01:34.657931076 +0000
sindhuja@SINDHUJA:~/practical_week10$ find . -inum 55670
./original.txt
sindhuja@SINDHUJA:~/practical_week10$ ln original.txt hardlink.txt
sindhuja@SINDHUJA:~/practical_week10$ ls -li original.txt hardlink.txt
55670 -rw-r--r-- 2 sindhuja sindhuja 83 Sep 27 13:02 hardlink.txt
55670 -rw-r--r-- 2 sindhuja sindhuja 83 Sep 27 13:02 original.txt
sindhuja@SINDHUJA:~/practical_week10$ ln -s original.txt symlink.txt
sindhuja@SINDHUJA:~/practical_week10$ ls -li original.txt symlink.txt
55670 -rw-r--r-- 2 sindhuja sindhuja 83 Sep 27 13:02 original.txt
55669 lrwxrwxrwx 1 sindhuja sindhuja 12 Sep 27 13:04 symlink.txt -> original.txt
sindhuja@SINDHUJA:~/practical_week10$ cat hardlink.txt
Operating System and Shell Programming
Practical Week 10
Linux Inode Demonstration
sindhuja@SINDHUJA:~/practical_week10$ cat symlink.txt
Operating System and Shell Programming
Practical Week 10
Linux Inode Demonstration
sindhuja@SINDHUJA:~/practical_week10$ |
```

Observation:

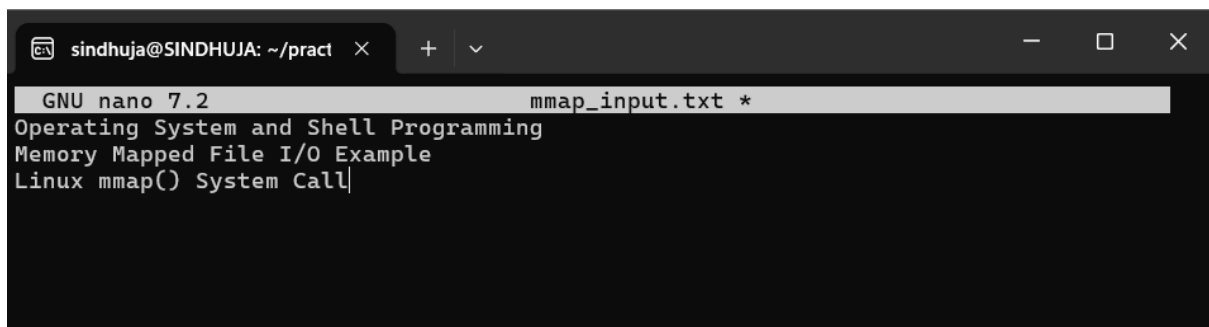
1. ls -li displayed inode numbers of files.
2. stat displayed detailed inode information.
3. find -inum searched files using inode numbers.
4. Hard links shared the same inode as the original file.
5. Symbolic links had a different inode and pointed to the original file.

Result:

Successfully investigated inode structures, created hard links and symbolic links, and observed the difference in inode allocation in Linux.

Aim:

To implement file reading and writing using the mmap() system call and compare memory-mapped I/O with traditional read/write operations in Linux.



```
sindhuja@SINDHUJA: ~/pract x + v
GNU nano 7.2 mmap_input.txt *
Operating System and Shell Programming
Memory Mapped File I/O Example
Linux mmap() System Call
```

```
sindhuja@SINDHUJA: ~/pract × + v
GNU nano 7.2 practical10_task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>

int main()
{
    int fd;
    struct stat fileStat;
    char *fileData;

    fd = open("mmap_input.txt", O_RDWR);

    if (fd < 0)
    {
        printf("Error opening file.\n");
        return 1;
    }

    fstat(fd, &fileStat);

    fileData = mmap(NULL, fileStat.st_size,
                    PROT_READ | PROT_WRITE,
                    MAP_SHARED, fd, 0);

    if (fileData == MAP_FAILED)
    {
        printf("Memory mapping failed.\n");
        close(fd);
        return 1;
    }

    printf("File Content Before Modification:\n%s\n", fileData);

    fileData[0] = 'o';

    msync(fileData, fileStat.st_size, MS_SYNC);

    printf("File Modified Successfully using mmap().\n");

    munmap(fileData, fileStat.st_size);
    close(fd);

    return 0;
}
```

```
sindhuja@SINDHUJA:~/practical_week10$ nano mmap_input.txt
sindhuja@SINDHUJA:~/practical_week10$ nano practical10_task2.c
sindhuja@SINDHUJA:~/practical_week10$ gcc practical10_task2.c -o practical10_task2
sindhuja@SINDHUJA:~/practical_week10$ ./practical10_task2
File Content Before Modification:
Operating System and Shell Programming
Memory Mapped File I/O Example
Linux mmap() System Call

File Modified Successfully using mmap().
sindhuja@SINDHUJA:~/practical_week10$ cat mmap_input.txt
operating System and Shell Programming
Memory Mapped File I/O Example
Linux mmap() System Call
```

```
sindhuja@SINDHUJA: ~/pract  × + ∨
GNU nano 7.2 compare_io.txt *
Traditional Read/Write I/O
- Uses read() and write() system calls.
- Data is copied between kernel space and user space.
- Suitable for sequential file access.

Memory-Mapped I/O
- Uses mmap() system call.
- File is mapped directly into memory.
- Faster for large files and random access.
- Reduces extra copying of data.

sindhuja@SINDHUJA:~/practical_week10$ nano compare_io.txt
sindhuja@SINDHUJA:~/practical_week10$ cat compare_io.txt
Traditional Read/Write I/O
- Uses read() and write() system calls.
- Data is copied between kernel space and user space.
- Suitable for sequential file access.

Memory-Mapped I/O
- Uses mmap() system call.
- File is mapped directly into memory.
- Faster for large files and random access.
- Reduces extra copying of data.
sindhuja@SINDHUJA:~/practical_week10$ |
```

Observation:

1. mmap() maps a file directly into memory.
2. Changes made through mapped memory are written back to the file.
3. Traditional I/O uses read() and write() system calls.
4. Memory-mapped I/O reduces extra data copying.
5. mmap() is more efficient for large files and random access.

Result:

Successfully implemented memory-mapped file I/O using mmap(), verified file modification, and compared it with traditional read/write file operations.